

# 程序报告

学号：2312331

姓名：王一诺

## 一、问题重述

(简单描述对问题的理解，从问题中抓住主干，必填)

1.1 实验背景

垃圾短信 (Spam Messages, SM) 是指未经过用户同意向用户发送不愿接收的商业广告或者不符合法律规范的短信。随着手机的普及，垃圾短信在日常生活日益泛滥，已经严重的影响到了人们的正常生活娱乐，乃至社会的稳定。据 360 公司 2020 年第一季度有关手机安全的报告提到，360 手机卫士在第一季度共拦截各类垃圾短信约 34.4 亿条，平均每日拦截垃圾短信约 3784.7 万条。大数据时代的到来使得大量个人信息数据得以沉淀和积累，但是庞大的数据量缺乏有效的整理规范；在面对量级如此巨大的短信数据时，为了保证更良好的用户体验，如何从数据中挖掘出更多有意义的信息为人们免受垃圾短信骚扰成为当前亟待解决的问题。

1.2 实验要求

1>任务提供包括数据读取、基础模型、模型训练等基本代码

2>参赛选手需完成核心模型构建代码，并尽可能将模型调到最佳状态

3>模型单次推理时间不超过 10 秒

1.3 实验环境

可以使用基于 Python 的 Pandas、Numpy、Sklearn 等库进行相关特征处理，使用 Sklearn 框架训练分类器，也可编写深度学习模型，使用过程中请注意 Python 包（库）的版本。

## 二、设计思想

(所采用的方法，有无对方法加以改进，该方法有哪些优化方向（参数调整，框架调整，或者指出方法的局限性和常见问题），伪代码，理论结果验证等... 思考题，非必填)

1.数据准备与预处理

该模型设计首先注重数据的平衡性和质量，通过将正负样本按 1:1 比例随机抽样，解决了类别不平衡问题。停用词处理采用了四川大学停用词库，有效过滤了无意义词汇。数据被随机划分为 80%训练集和 20%测试集，确保模型评估的可靠性。

2.特征工程

采用 TF-IDF 向量化方法将文本转换为数值特征，并使用 1-gram 和 2-gram 组合(n-gram 范围 (1,2))，既考虑了单个词的重要性，又捕捉了词语组合的语义信息。通过 MaxAbsScaler 进行归一化。

3.算法选择

在测试集上的混淆矩阵:

```
[[14660 1144]
 [ 98 15757]]
```

在测试集上的分类结果报告:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.99      | 0.93   | 0.96     | 15804   |
| 1            | 0.93      | 0.99   | 0.96     | 15855   |
| accuracy     |           |        | 0.96     | 31659   |
| macro avg    | 0.96      | 0.96   | 0.96     | 31659   |
| weighted avg | 0.96      | 0.96   | 0.96     | 31659   |

在测试集上的 f1-score :  
0.9620832824520699

在测试集上的混淆矩阵:

```
[[14637 1046]
 [ 101 15875]]
```

在测试集上的分类结果报告:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.99      | 0.93   | 0.96     | 15683   |
| 1            | 0.94      | 0.99   | 0.97     | 15976   |
| accuracy     |           |        | 0.96     | 31659   |
| macro avg    | 0.97      | 0.96   | 0.96     | 31659   |
| weighted avg | 0.97      | 0.96   | 0.96     | 31659   |

在测试集上的 f1-score :  
0.965133598808402

对 MultinomialNB (如左上图所示) 和 ComplementNB (如右上图所示) 都进行了测试, 差别不大, 最终选择了 ComplementNB(补充朴素贝叶斯)作为分类器, 这是对传统多项式朴素贝叶斯的改进, 特别适合处理不平衡数据集和 TF-IDF 特征。该算法通过调整权重计算方式, 能更好地处理某些类别中特征缺失的情况。

#### 4.工程实现

采用 Scikit-learn 的 Pipeline 将特征转换、归一化和分类器串联起来, 形成端到端的处理流程, 避免了数据泄露问题, 同时提高了代码的可维护性和可重复性。模型训练完成后, 使用 joblib 进行序列化保存, 便于后续对新数据的预测。

### 三、代码内容

(能体现解题思路的主要代码, 有多个文件或模块可用多个"===="隔开, 必填)

```
=====train.py=====

import os
os.environ["HDF5_USE_FILE_LOCKING"] = "FALSE"
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
# ----- 停用词库路径, 若有变化请修改 -----
stopwords_path = r'scu_stopwords.txt'
data_path = "./datasets/5f9ae242cae5285cd734b91e-momodel/sms_pub.csv"
# -----
sms = pd.read_csv(data_path, encoding='utf-8')
sms_pos = sms[(sms['label'] == 1)]
sms_neg = sms[(sms['label'] == 0)].sample(frac=1.0)[: len(sms_pos)]
sms = pd.concat([sms_pos, sms_neg], axis=0).sample(frac=1.0)

def read_stopwords(stopwords_path):
    """
    读取停用词库
    :param stopwords_path: 停用词库的路径
    :return: 停用词列表, 如 ['嘿', '很', '乎', '会', '或']
    """
    stopwords = []
    # ----- 请完成读取停用词的代码 -----
    with open(stopwords_path, 'r', encoding='utf-8') as f:
```

```

        stopwords = f.read()
        stopwords = stopwords.splitlines()
        #-----
        return stopwords
# 读取停用词
stopwords = read_stopwords(stopwords_path)

X = np.array(sms.msg_new)
y = np.array(sms.label)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=22, test_size=0.2)
print("the number of all the datas", X.shape)
print("the number of the train datas", X_train.shape)
print("the number of the test datas", X_test.shape)

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.preprocessing import MaxAbsScaler

# pipeline_list 用于传给 Pipeline 作为参数
pipeline_list = [
    # ----- 需要完成的代码 -----
    ('tfidf', TfidfVectorizer(stop_words=stopwords, ngram_range=(1,2))),
    ('MaxAbsScaler', MaxAbsScaler()),
    ('classifier', ComplementNB())
    # -----
]

# 搭建 pipeline
pipeline = Pipeline(pipeline_list)

# 训练 pipeline
pipeline.fit(X_train, y_train)

# 对测试集的数据集进行预测
y_pred = pipeline.predict(X_test)

# 在测试集上进行评估
from sklearn import metrics
print("在测试集上的混淆矩阵: ")
print(metrics.confusion_matrix(y_test, y_pred))
print("在测试集上的分类结果报告: ")

```

```

print(metrics.classification_report(y_test, y_pred))
print("在测试集上的 f1-score : ")
print(metrics.f1_score(y_test, y_pred))

# 在所有的样本上训练一次，充分利用已有的数据，提高模型的泛化能力
pipeline.fit(X, y)
# 保存训练的模型，请将模型保存在 results 目录下
import joblib
pipeline_path = 'results/pipeline_ComplementNB.model'
joblib.dump(pipeline, pipeline_path)

=====main.py=====

import os
os.environ["HD5_USE_FILE_LOCKING"]="FALSE"
import joblib
stopwords_path = r'scu_stopwords.txt'
def read_stopwords(stopwords_path):
    stopwords=[]
    with open(stopwords_path,'r',encoding='utf-8') as f:
        stopwords=f.read()
        stopwords=stopwords.splitlines()
    return stopwords

stopwords=read_stopwords(stopwords_path)

# 加载训练好的模型
pipeline_path = 'results/pipeline_ComplementNB.model'
pipeline = joblib.load(pipeline_path)

def predict(message):
    """
    预测短信短信的类别和每个类别的概率
    param: message: 经过 jieba 分词的短信,如"医生 拿 着 我 的 报告单 说 : 幸亏 你 来 的
    早 啊"
    return: label: 整数类型，短信的类别，0 代表正常，1 代表恶意
           proba: 列表类型，短信属于每个类别的概率，如[0.3, 0.7]，认为短信属于 0 的概率
    率为 0.3，属于 1 的概率为 0.7
    """
    label = pipeline.predict([message])[0]
    proba = list(pipeline.predict_proba([message])[0])

    return label, proba

```

#### 四、实验结果

(实验结果，必填)

## 1>训练模型

在测试集上的混淆矩阵:

```
[[14637 1046]
 [ 101 15875]]
```

在测试集上的分类结果报告:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.99      | 0.93   | 0.96     | 15683   |
| 1            | 0.94      | 0.99   | 0.97     | 15976   |
| accuracy     |           |        | 0.96     | 31659   |
| macro avg    | 0.97      | 0.96   | 0.96     | 31659   |
| weighted avg | 0.97      | 0.96   | 0.96     | 31659   |

在测试集上的 f1-score :

0.965133598808402

## 2>实例测试

### 接口测试

✓ 接口测试通过。

### 用例测试

| 测试点                  | 状态 | 时长  | 结果                                  |
|----------------------|----|-----|-------------------------------------|
| 测试模型<br>预测结果         | ✓  | 13s | 通过测试，训练的分类器具备检测恶意短信的能力，分类正确比例:10/10 |
| 测试读取<br>停用词库<br>函数结果 | ✓  | 13s | read_stopwords 函数返回的类型正确            |

## 五、总结

（自评分析（是否达到目标预期，可能改进的方向，实现过程中遇到的困难，从哪些方面可以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

- 1.本实验通过采用朴素贝叶斯分类器和 TF-IDF 特征提取方法，实现了对垃圾短信的识别。
- 2.通过实验，熟悉了 py 搭建模型、训练和评估模型以及使用模型预测新数据的操作。
- 3.在实验中意识到，数据预处理是 NLP 乃至所有项目的重中之重。比如本次实验中训练样本关于判断指标分布极不均衡，就要考虑到降采样或者重采样的方法平衡各类别训练数据。